📘 Answer Key — XII Computer Science (083)

Unit Test – 2 (UT-2)

Section A 1 mark each

Q.No	Answer
1	False
2	b) wb+
3	a) Push
4	b) Attribute
5	b) readline()
6	d) Both b) and c)
7	a) The code will create a CSV file named 'data.csv' with two rows.
8	b) DISTINCT
9	c) pickle
10	b) DESCRIBE STUDENT;
11	b) Write mode
12	a) 4
13	b) stack.append(item)
14	c) HAVING

Section B 2 marks each

15. A. read() vs readlines()

read()	readlines()
Returns entire file as a single string.	Returns a list of lines (strings).
`f.read() → 'Hello\nWorld'`	`f.readlines() → ['Hello\n', 'World']`
Optional size: `f.read(n)`	Optional hint: `f.readlines(n)`

15. B. Primary Key & Foreign Key

- **Primary Key:** Uniquely identifies each row; cannot be NULL. *Example:* `Roll_No` in `STUDENT`.
- **Foreign Key:** Refers to primary key of another table; creates link. *Example:* `Roll_No` in `MARKS` references `STUDENT(Roll_No)`.

16. Output

```
brown fox
```

17. A. Import pickle & read object

```
import pickle
with open("data.dat", "rb") as f:
    obj = pickle.load(f)
```

17. B. CREATE TABLE EMPLOYEE

```
CREATE TABLE EMPLOYEE (
    Emp_ID INTEGER PRIMARY KEY,
    Emp_Name VARCHAR(50) NOT NULL,
    Salary FLOAT
);
```

18. Pop operation on stack

```
stack = [10, 20, 30]

def pop_stack():
    if len(stack) == 0:
        print("Stack Underflow")
    else:
        return stack.pop()

print(pop_stack())   # 30
```

Section C 3 marks each

19. A. count_the()

```
def count_the():
    with open("ARTICLE.TXT", "r") as f:
        content = f.read()
    words = content.lower().split()
    count = words.count("the")
    print("Number of times 'the' appears:", count)
```

19. B. SQL queries on PRODUCT

```
-- I. Products with quantity between 10 and 50
SELECT P_Name, Price FROM PRODUCT WHERE Qty BETWEEN 10 AND 50;

-- II. Average price of all products
SELECT AVG(Price) FROM PRODUCT;

-- III. Maximum price for each category
SELECT Category, MAX(Price) FROM PRODUCT GROUP BY Category;
```

20. A. Employee stack

```
employees = [("E1","Alice",60000),("E2","Bob",45000),
             ("E3","Charlie",70000),("E4","Diana",50000)]
Emp_Stack = []

def Push_emp():
    for emp in employees:
        if emp[2] > 50000:
            Emp_Stack.append(emp)

def Pop_emp():
    if len(Emp_Stack) == 0:
        print("Stack is Empty")
    else:
        while Emp_Stack:
            print(Emp_Stack.pop())

Push_emp()
Pop_emp()
```

20. B. add_records()

```
import csv

def add_records(data):
    with open("records.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Name", "Age", "City"])
        writer.writerows(data)
```

Section D 4 marks each

21. A. SALES.csv operations

```
import csv

def append_record():
    pid = input("Enter Product_ID: ")
    pname = input("Enter Product_Name: ")
    qty = int(input("Enter Quantity: "))
    price = float(input("Enter Price: "))
    with open("SALES.csv", "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([pid, pname, qty, price])

def calculate_total_revenue():
    total = 0
    with open("SALES.csv", "r") as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            total += float(row[2]) * float(row[3])
    return total
```

21. B. Even-length lines

```
count = 0
with open("DATA.TXT", "r") as fin, open("EVEN.TXT", "w") as fout:
    for line in fin:
        if len(line.rstrip("\n")) % 2 == 0:
            fout.write(line)
            count += 1
print("Total lines written to EVEN.TXT:", count)
```

22. SQL queries on BOOKS and AUTHORS

```
-- I. Fiction books
SELECT B.Title, A.Author_Name, B.Price
FROM BOOKS B, AUTHORS A
WHERE B.Author_ID = A.Author_ID AND B.Genre = 'Fiction';

-- II. Country & total books
SELECT A.Country, COUNT(*) AS Total_Books
FROM BOOKS B, AUTHORS A
WHERE B.Author_ID = A.Author_ID
GROUP BY A.Country;

-- III. Above average price
SELECT B.Title, A.Author_Name
FROM BOOKS B, AUTHORS A
WHERE B.Author_ID = A.Author_ID
  AND B.Price > (SELECT AVG(Price) FROM BOOKS);

-- IV. Sorted by Author ASC, Price DESC
SELECT B.Title, A.Author_Name, B.Price
FROM BOOKS B, AUTHORS A
WHERE B.Author_ID = A.Author_ID
ORDER BY A.Author_Name ASC, B.Price DESC;
```

23. SQL queries on STUDENTS and EXAMS

```
-- I. Cartesian Product
SELECT * FROM STUDENTS, EXAMS;

-- II. Equi-join
SELECT S.Student_Name, E.Subject, E.Score
FROM STUDENTS S, EXAMS E
WHERE S.Roll_No = E.Roll_No;

-- III. Score > 90
SELECT S.Student_Name, E.Subject, E.Score
FROM STUDENTS S, EXAMS E
WHERE S.Roll_No = E.Roll_No AND E.Score > 90;

-- IV. Average > 85
SELECT S.Student_Name, AVG(E.Score) AS Avg_Score
FROM STUDENTS S, EXAMS E
WHERE S.Roll_No = E.Roll_No
GROUP BY S.Student_Name
HAVING AVG(E.Score) > 85;
```

Section E 5 marks each

24. Log file functions

```
def count_error_lines():
    count = 0
    with open("LOG.txt", "r") as f:
        for line in f:
            if "ERROR" in line:
                count += 1
    return count

def create_error_report():
    with open("LOG.txt", "r") as fin, open("WARNINGS.txt", "w") as fout:
        for line in fin:
            if "WARNING" in line:
                fout.write(line)

def find_keyword_position(keyword):
    with open("LOG.txt", "r") as f:
        content = f.read()
        pos = content.find(keyword)
        if pos != -1:
            f.seek(pos)
            print("Keyword found at position:", f.tell())
        else:
            print("Keyword not found")
```

25. A. Book stack functions

```
book_stack = []

def push_book():
    bid = input("Enter Book_ID: ")
    title = input("Enter Book_Title: ")
    author = input("Enter Author: ")
    price = float(input("Enter Price: "))
    stock = int(input("Enter Stock: "))
    book_stack.append((bid, title, author, price, stock))
    print("Book pushed.")

def pop_book():
    if len(book_stack) == 0:
        print("Stack is Empty")
    else:
        print("Popped:", book_stack.pop())

def display_stack():
    if len(book_stack) == 0:
        print("Stack is Empty")
    else:
        for book in reversed(book_stack):
            print(book)
```

25. B. SQL query outputs

I. IT employees

Emp_Name	Salary
Raj Sharma	75000
Sneha Reddy	80000
Sunil Verma	90000

II. Department & average salary

Department	AVG(Salary)
IT	81666.67
HR	51500.00
Finance	65000.00
Marketing	60000.00

III. Emp_Name LIKE 'S%'

Emp_ID	Emp_Name	Department	Salary	Joining_Date
E104	Sneha Reddy	IT	80000	2018-11-25
E106	Sunil Verma	IT	90000	2020-07-12

IV. COUNT(*) Joining_Date > '2020-01-01'

```
5
```

V. Department summary

Department	Total_Employees	Total_Salary
IT	3	245000
HR	2	103000
Finance	1	65000
Marketing	1	60000